

Sprint 1 – Manager with Reserve

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: ☐ Yes ☐ No

Name:

Date:

Submission Details

Final **Changelist** number:

Verified build: ☐ Yes ☐ No

Required Configurations:

DEBUG only

YouTubeLink:

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.gls, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
 - Design Patterns – no minimum
 - Sprints – minimum of 5 real submissions (generally 10+)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays)
- Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - NO arrays allowed!
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

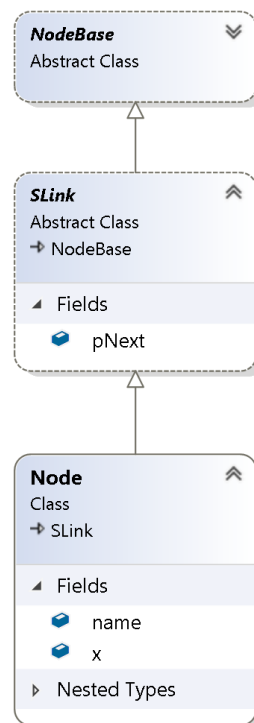
Goals

- Manager with Reserve using Single Linked List
 - A small twist on the demos from class
- Create a reserve manager using single linked lists
 - Add, Remove, Find methods
 - SLink manager is internally created inside the Manager.

Assignments

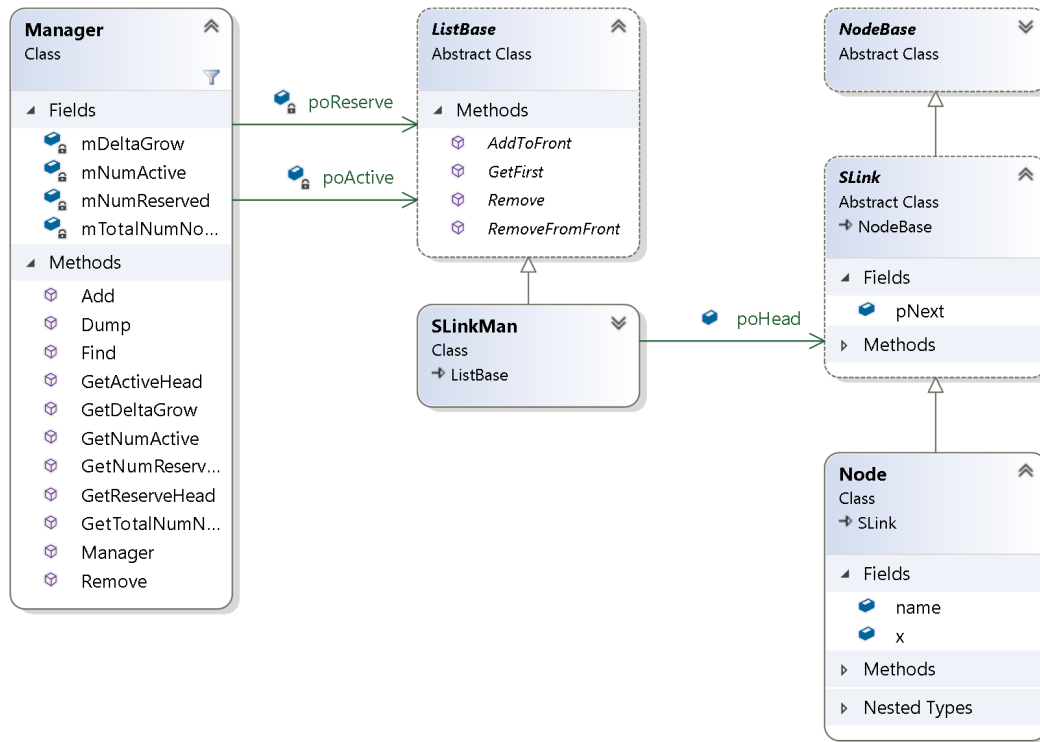
Grading

- Game Engine has a unit test in Main
- Make sure you pass all 4 tests.
- Code a reserve manager using the provided class, Node:



- Feel Free to add classes to the project
 - Modify the Class Diagram
 - Manager class is stubbed out
 - Please fill in the missing methods
 - Feel free to add more method and data

- Here is my UML diagram to give you some ideas:



- Do NOT modify the unit tests
 - Conform to the methods provided.
- You will need to add files to the project
 - My suggestion
 - Always create the directories and files in the tool
 - This way they are in the correct location
 - Then cut and paste into those files
- Namespace
 - Keep every file using namespace SpaceInvaders
- Demonstrate you have all the tools and setup working

Required Features

Manager

- Create a Manager
 - Manager actually has 2 linked list attached an Active List and a Reserved List
 - essentially 2 head pointers
 - All nodes are single linked lists
- Manager(...) - Initialized you manager with a reserved size and growth size
 - Example, say you initialize the reserve size with 5 nodes, growth size is 2 nodes
 - You will create on construction of the manager
 - 5 linked nodes attached to the reserved list.

- You set the growth size member variable to 2
- The Active list will have nothing attached only null pointer.
- Generally
 - Adding a node
 - You move a node from the reserved list and add it to the active list
 - Insert the node to the front of the list
 - In front for speed.... Think about it $O(1)$
 - If the reserved list is empty
 - Refill the reserve list with the growth size
 - The reserved list will be filled with that many nodes, then you can move one from the reserve and transfer it to the active list
 - (might seem unnecessary to move to reserve then immediately transfer to active... but it reduces complexity)
 - Remove a node
 - Remove any given node by address
 - Worse case $O(N)$ – it's a single linked list
 - Add it back to the Reserve List
 - To the front of the list $O(1)$ complexity
- Add(...) - creates and return node
 - moves one from the reserved list and inserts the node to the active list
- Remove(...) - recycles the node
 - removes the node from active list and transfers it to the reserved list
- Ability to report stats – data we can use
 - number of active, reserved, growth number, total number, etc...
- Find(...) - searches through active list and returns node
 - Search by Name
 - Hint – you'll need to have casting to make this possible

Node

- Holds data (int x)
- Holds name (enumeration)
- Inherits from SLink (single linked list)

SLink

- Holds the next references
- Node derives from SLink
- Used for implementing in terms of... paradigm

NodeBase

- Abstract base class (place holder)
- Do nothing in this class

Video Capture

- Have Visual Studio open and ready to go.
- Start recording
 - Show compiling and output window running the tests (All 4 test passing)
- Commentary:
 - Speak clearly and loud
 - Show off a section of the code (suggestion the Find() function)
 - Show the UML diagram (in the tool)
 - Show your code and quickly explain it.
 - Duration
 - Approx. 2 min long YouTube video capture
 - Do not exceed 5 min
 - Use Zoom recording
 - High resolution
 - Good sound quality
 - Show on the capture... it compiling and then running
 - Place YouTube link in PDF
 - Make sure it public and runs without passwords or login
 - Watch your video
 - Can you see the code clearly
 - Can you hear your voice clearly
- Don't lose points
 - Make sure everything is submitted properly
 - Don't record beyond 5 min.
 - Your audio volume was too quiet (test it and verify it)
 - Your YouTube link needs to work "As-IS" with no login
 - Don't make it private... make it unlisted
 - You check-in too many files or temporary files
 - Make sure you run CleanMe.bat before perform submission

General guidelines:

- Post on Piazza questions and clarifications
- Make sure you delete these using directives (we are not using them)
 - `using System.Collections.Generic;`
 - `using System.Linq;`
 - `using System.Text;`
 - `using System.Threading.Tasks;`

Development

- Use the project in Sprint1
- Do not copy temporary files or submit extra data
- Make sure your run CleanMe.bat before adding to performce

Submission

- Submit your SprintX directory into performce:
 - /student/<yourname>/SprintX/...
 - You need to submit a complete C# project
 - Solution, project and C# files (whatever it takes to build the project)
 - Do not submit anything that is auto generated
 - Run the supplied CleanMe.bat before submission
 - Should cleanup files
- Fill out the Submission report and submit that pdf to your student directory

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performce?
- Follow the verification process for performce
 - Is all the code there and compiles “as-is”?
 - No extra files
- Submit the YouTube link to PDF

Hints

Most assignments will have hints in a section like this.

- Reuse the work you did in the Linked List assignment
- Make sure you are using Single Linked List
- Study the projects in class lecture

Troubleshooting

- Print to the output window to help you debug
 - It really helps
- Ask for clarification on piazza